API SECURITY RISK ANALYSIS

Future Interns Cyber Security Internship

Task 3: Security Assessment Using Postman

Submitted by: Harshitha R Shet

Department: Computer Science and Engineering

Tool Used: Postman

API Tested: JSON Placeholder Public Demo API

Project Type: API Security Assessment and Documentation

Submission Date: 21 October 2026

INTRODUCTION AND OBJECTIVES

1. Introduction

Application Programming Interfaces (APIs) enable different software applications to communicate and exchange data. APIs are widely used in web applications, mobile applications, cloud platforms, and enterprise systems.

API security is important because improperly protected APIs may expose sensitive information, allow unauthorized access, or be abused through excessive requests and inadequate input validation.

This project focuses on examining the responses and observable security-related headers of a public demonstration API using Postman. The assessment uses read-only HTTP GET requests to observe endpoint behavior, returned data, authentication settings, and rate-limit-related headers.

The project is intended for educational purposes. The observations are limited to the requests and responses captured during testing and do not represent a complete security audit of the API.

2. Objectives

The main objectives of this task are:

- To understand API requests and HTTP response status codes.
- To inspect API responses using Postman.
- To observe endpoint behavior with and without authentication credentials.
- To examine the fields returned by API endpoints.
- To review rate-limit-related response headers.
- To understand the importance of input validation and access control.
- To document findings and recommend appropriate security practices.

3. Scope of Assessment

The assessment covers the following areas:

1. Authentication and authorization observations.
2. Data exposure and data minimization.
3. Rate-limit-related response headers.

4. Input handling and endpoint behavior.
5. Security recommendations and limitations.

Only the public JSONPlaceholder demonstration API was used. No destructive operations or high-volume requests were performed.

TOOLS AND METHODOLOGY

4. Tools and Technologies Used

4.1 Postman

Postman is an API testing tool used to send HTTP requests and inspect responses. It provides information such as status codes, response bodies, headers, and request configuration.

4.2 JSONPlaceholder

JSONPlaceholder is a public demonstration REST API that provides sample resources such as users and posts. It was selected to practice API request handling and response analysis.

4.3 HTTP GET Method

The GET method is used to retrieve information from an API endpoint. It was used for the requests in this assessment.

5. API Endpoints Tested

The following endpoints were examined:

Endpoint 1: Retrieve users

GET https://jsonplaceholder.typicode.com/users

Purpose: To observe the response containing a collection of sample user records.

Endpoint 2: Retrieve an existing user

GET https://jsonplaceholder.typicode.com/users/1

Purpose: To inspect the response for an individual sample user.

Endpoint 3: Retrieve a nonexistent user

GET https://jsonplaceholder.typicode.com/users/9999

Purpose: To observe how the API responds when the requested user record does not exist.

6. Testing Methodology

The following procedure was used:

1. Open Postman and create a GET request.
2. Enter the endpoint URL.
3. Select the No Auth authorization setting.
4. Send the request and record the HTTP status code.
5. Inspect the response body and returned fields.
6. Review response headers for security-related information.
7. Compare the results from the different endpoints.
8. Document observations, limitations, and recommendations.

The assessment was conducted using a small number of read-only requests. The results reflect the observed responses rather than assumptions about the API's internal implementation.

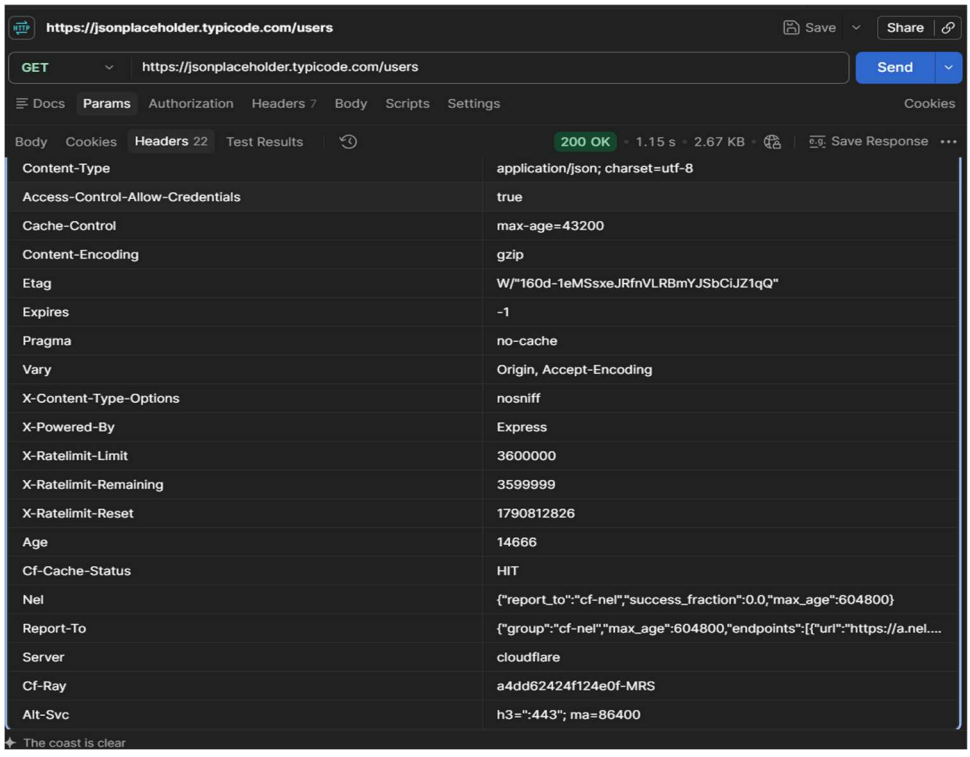


Figure 1: Successful GET request to the users endpoint in Postman.

Use the screenshot showing GET /users with the 200 OK status and the response headers. If the screenshot becomes too small to read, use the version with the clearest text.

AUTHENTICATION AND DATA EXPOSURE ANALYSIS

7.Authentication and Authorization Assessment

7.1 Test Description

The Authorization tab in Postman showed the No Auth setting for the tested requests. A GET request to /users/1 returned a successful response with HTTP status 200 OK.

A request to /users/9999 returned HTTP status 404 Not Found.

7.2 Observed Results

The public demonstration endpoint returned sample user data without requiring authorization credentials. The nonexistent user request returned a not-found response.

7.3 Analysis

The absence of authentication on a public demonstration endpoint is not, by itself, a security vulnerability. Public APIs may intentionally allow unauthenticated access to sample resources.

The 404 Not Found response indicates that the requested resource was not found. This is expected behavior for a nonexistent record and does not demonstrate unauthorized access.

In a production application, authentication and authorization should be implemented according to the sensitivity of the data and the intended access permissions.

7.4 Recommendation

Production APIs should require authentication for private resources and enforce authorization checks for each protected resource. Sensitive information should not be accessible merely by changing a resource identifier.

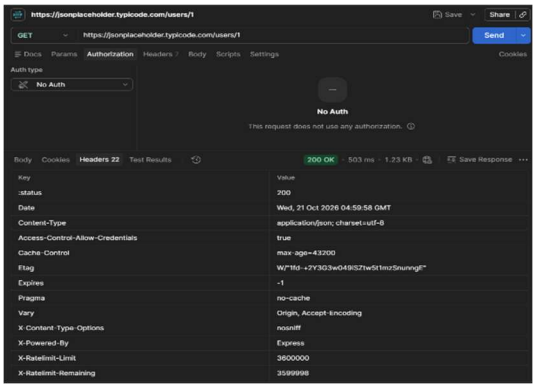


Figure 2: *GET request to /users/1 returning HTTP 200 OK and a sample user record.*

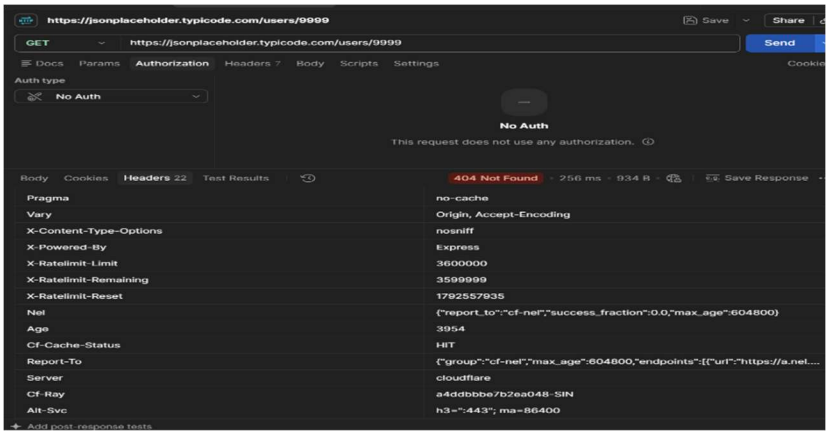


***Figure 3**: GET request to /users/9999 returning HTTP 404 Not Found with No Auth selected.*

8. Data Exposure Assessment

8.1 Test Description

The response body for /users/1 was examined to identify the fields returned by the API.

8.2 Observed Fields

The response included sample fields such as:

- User ID
- Name and username
- Email address
- Address and geographical coordinates
- Phone number
- Website
- Company information

8.3 Analysis

The demonstration API returns fabricated sample information for educational and testing purposes. The presence of these fields does not establish a data exposure vulnerability in this demo.

In a real production application, returning unnecessary personal or confidential information may create privacy and security risks if the information is accessible to unauthorized users.

8.4 Recommendation

API developers should apply data minimization by returning only the fields required for the intended operation. Access to sensitive information should be restricted through appropriate authentication and authorization controls.

RATE LIMITING AND INPUT VALIDATION

9. Rate-Limiting Assessment

9.1 Objective

Rate limiting restricts the number of requests a client can make within a defined period. It can help protect services from excessive traffic, automated abuse, and certain denial-of-service scenarios.

9.2 Observed Results

The response headers captured in Postman included the following values:

- X-Ratelimit-Limit: 3600000
- X-Ratelimit-Remaining: 3599999 in the collection request screenshot
- X-Ratelimit-Remaining: 3599998 in the individual user request screenshot
- X-Ratelimit-Reset: 1790812826 in the captured headers

The X-Content-Type-Options header was also present with the value nosniff.

9.3 Analysis

The captured responses contain rate-limit-related headers. However, the exact meaning, applicable time window, and enforcement behavior of these values were not independently verified.

The available evidence is therefore insufficient to determine whether the API enforces an effective rate limit under different request conditions. No high-volume testing was performed.

The X-Content-Type-Options: nosniff header instructs compatible browsers not to infer a different content type from the declared type. Its presence is a positive security-related observation, but it does not establish the overall security of the API.

9.4 Recommendation

Production APIs should implement and document appropriate server-side request limits, monitor repeated requests, and return HTTP 429 Too Many Requests when a client exceeds an enforced limit.

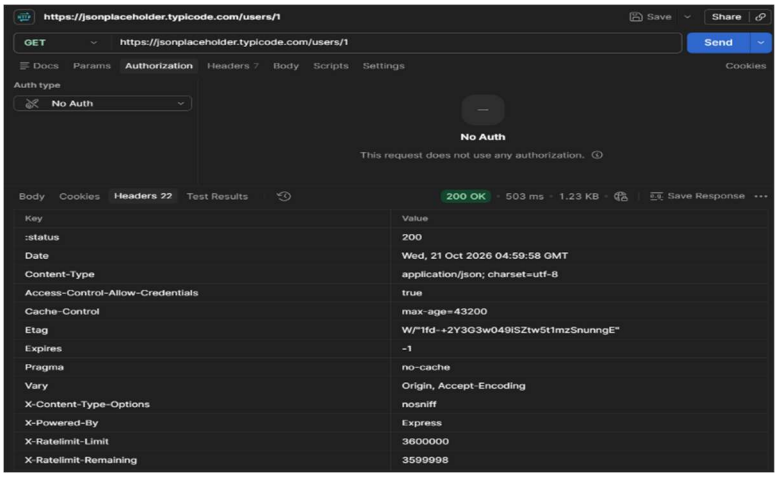


Figure 4: Postman response headers showing rate-limit-related values and the X-Content-Type-Options header.

Use the screenshot where X-Ratelimit-Limit, X-Ratelimit-Remaining, X-Ratelimit-Reset, and X-Content-Type-Options are visible.

10. Input Validation Assessment

10.1 Objective

Input validation helps ensure that API requests handle supplied parameters and identifiers appropriately.

10.2 Observed Behavior

The request to /users/9999 returned HTTP 404 Not Found, while the request to /users/1 returned HTTP 200 OK.

10.3 Analysis and Limitation

These results demonstrate different responses for an existing and a nonexistent resource identifier. They do not establish whether the API validates every possible input correctly.

The assessment did not establish whether malformed inputs, unexpected data types, or invalid parameters are handled securely in all cases.

10.4 Recommendation

Production APIs should validate input types, formats, lengths, and permitted values on the server side. Invalid input should be handled consistently without exposing sensitive implementation details.

FINDINGS, RECOMMENDATIONS AND CONCLUSION

11. Summary of Findings

Assessment area	Observation	Conclusion
Authentication	Public GET requests succeeded without credentials	Expected for the tested demo endpoint
Resource handling	/users/1 returned 200 OK; /users/9999 returned 404 Not Found	Expected endpoint behavior
Data exposure	Sample user records included contact and address fields	Data minimization should be considered in production
Rate limiting	Rate-limit-related headers were present	Enforcement and exact semantics remain unverified
Security headers	X-Content-Type-Options: nosniff was present	Positive observation; not a complete security assessment
Input validation	Different responses were observed for existing and nonexistent identifiers	Insufficient evidence for a complete validation assessment

No confirmed security vulnerability was established through the tests performed.

12. General Security Recommendations

The following practices are recommended for production API systems:

1. **Authentication:** Require appropriate authentication for protected resources.
2. **Authorization:** Enforce permissions for each resource and operation.
3. **Data minimization:** Return only the information needed by the client.
4. **Rate limiting:** Apply server-side request limits and monitor abnormal traffic.
5. **Input validation:** Validate request parameters and reject invalid inputs safely.
6. **Security headers:** Configure appropriate headers based on the application's architecture and threat model.
7. **Logging and monitoring:** Record relevant security events while avoiding unnecessary sensitive information in logs.

8. **Documentation:** Maintain clear API documentation describing authentication requirements, error responses, and usage limits.

13. Limitations

This assessment was limited to a public demonstration API and a small number of read-only GET requests. The results reflect only the responses and headers visible during testing.

The assessment did not include source-code review, authenticated testing, authorization bypass testing, load testing, or a comprehensive security audit. The exact meaning and enforcement of the observed rate-limit-related headers were not independently verified.

Consequently, the findings should not be interpreted as proof that the API is either fully secure or vulnerable.

14. Conclusion

This task provided practical experience in sending HTTP GET requests, interpreting response status codes, inspecting JSON response bodies, and reviewing response headers using Postman.

The assessment demonstrated successful retrieval of sample user data and a not-found response for a nonexistent user. It also identified rate-limit-related headers and the X-Content-Type-Options header in the captured responses.

The exercise improved my understanding of API security concepts, including authentication, authorization, data minimization, rate limiting, and input validation. Further testing on an authorized test environment would be necessary to evaluate these controls more comprehensively.

15. References

1. JSONPlaceholder — Public demonstration REST API: https://jsonplaceholder.typicode.com/
2. Postman Documentation: https://learning.postman.com/docs/
3. OWASP API Security Project: https://owasp.org/www-project-api-security/
4. OWASP API Security Top 10: https://owasp.org/API-Security/

End of Report